

NumPy e computação numérica

Eduardo Adame

Ciência de Dados

20 de março de 2026



Por que NumPy?



Problema: Python puro não é eficiente para computação numérica.

Listing 1: Python vanilla

```
1 L = list(range(1000))  
2 [i**2 for i in L]
```

Listing 2: NumPy

```
1 import numpy as np  
2 a = np.arange(1000)  
3 a**2
```

- NumPy é **vetorizado**
- Executa operações em **C por baixo**
- Muito mais rápido e eficiente

O que é um array?



- Estrutura de dados homogênea
- Elementos do mesmo tipo
- Armazenamento contíguo em memória

Listing 3: Exemplo

```
1 a = np.array([1,2,3])  
2 a.shape  
3 a.dtype
```

Importante

NumPy não é uma lista melhor — é outra estrutura.



Listing 4: Criação

```
1 np.array([1,2,3])
2
3 np.arange(0,10)
4
5 np.linspace(0,1,5)
6
7 np.zeros((3,3))
8
9 np.ones((2,2))
10
11 np.eye(3)
```

- Evite construir arrays manualmente
- Prefira funções vetorizadas



Listing 5: Exemplo

```
1 a = np.array([[1,2,3],[4,5,6]])  
2  
3 a.shape      # (2,3)  
4 a.ndim       # 2
```

- shape = tamanho em cada dimensão
- ndim = número de dimensões



Listing 6: Indexação

```
1 a = np.arange(10)
2
3 a[0]
4 a[-1]
5 a[2:5]
6 a[:,2]
```

Listing 7: 2D

```
1 b = np.array([[1,2],[3,4]])
2
3 b[0,1]
```



Listing 8: Elementwise

```
1 a = np.array([1,2,3])
2
3 a + 1
4 a * 2
5 a ** 2
```

- Sem loops explícitos
- Muito mais rápido



Listing 9: Python

```
1 [i+1 for i in range(10000)]
```

Listing 10: NumPy

```
1 a = np.arange(10000)
2 a + 1
```

Evite loops sempre que possível.



Listing 11: Exemplo

```
1 a = np.array([[1,2,3],  
2               [4,5,6]])  
3  
4 b = np.array([1,2,3])  
5  
6 a + b
```

- NumPy expande dimensões automaticamente
- Evita loops explícitos



Listing 12: Exemplo

```
1 a = np.array([[1,2],[3,4]])  
2  
3 a.sum()  
4 a.sum(axis=0)  
5 a.mean()  
6 a.std()
```

- axis controla dimensão da operação



Listing 13: Comparações

```
1 a = np.array([1,2,3,4])  
2  
3 a > 2  
4 a == 3
```

Listing 14: Máscaras booleanas

```
1 mask = a > 2  
2 a[mask]
```

Listing 15: Filtro direto

```
1 a[a > 2]
```



Listing 16: Combinações

```
1 (a > 1) & (a < 4)
2 (a == 1) | (a == 4)
```

- Permite selecionar subconjuntos de dados
- Base para **data filtering** em ciência de dados
- Evita loops explícitos



Listing 17: Produto

```
1 A @ B
```

- `*` não é produto matricial
- use `@` ou `np.dot`



Listing 18: Benchmark

```
1 import time
2
3 initial_time = time.time()
4 # python
5 python_time = time.time() - initial_time
6 initial_time = time.time()
7 # numpy
8 numpy_time = time.time() - initial_time
```

- NumPy usa memória contígua
- operações vetorizadas
- evita overhead do Python

Obrigado!

Alguma dúvida?

Agora vamos para os exercícios!